Phase 3 — Core ML Skills

ExamGuard AI — Implementation Guide

Scikit-Learn - The ML Library

What is Scikit-Learn?

Scikit-Learn (often written as `sklearn`) is THE machine learning library for Python. It contains all the classic ML algorithms in one package, with a simple, consistent interface.

Think of it as a toolbox: you pick a model, give it data, and it learns.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)       # Train
predictions = model.predict(X_test) # Predict

# That's it. Three lines to train and use an ML model.
```

Why Scikit-Learn Matters for ExamGuard

Before Deep Learning, Start Simple

You DON'T jump straight to YOLO and CNN. First, you learn ML fundamentals with Scikit-Learn:

```
Learning path:
 Scikit-Learn (simple models, learn the workflow)
    ↓
 PyTorch/TensorFlow (deep learning models, same workflow but more powerful)
    ↓
 YOLO, CNN, LSTM (specialized models for ExamGuard)
```

What Scikit-Learn teaches you:

- How to train ANY model (the workflow is the same everywhere)
- How to evaluate models (accuracy, precision, recall)
- How to preprocess data (scaling, encoding)
- How to avoid common mistakes (overfitting, data leakage)

ExamGuard uses Scikit-Learn for:

- Initial experiments - Test if simple models can distinguish cheating from normal behavior
- Feature engineering - Create and test features before using deep learning
- Evaluation - All metrics (accuracy, precision, recall, F1) come from sklearn
- Data splitting - train_test_split is from sklearn
- Preprocessing - Scaling, normalizing data before feeding to any model
- Baselines - "Can a simple model do this? If yes, maybe we don't need deep learning."

What to Learn

1. The Universal ML Workflow

Every ML model in Scikit-Learn follows the SAME pattern:

```
# Step 1: Import the model
from sklearn.tree import DecisionTreeClassifier

# Step 2: Create the model
model = DecisionTreeClassifier()

# Step 3: Train the model
model.fit(X_train, y_train)

# Step 4: Make predictions
predictions = model.predict(X_test)

# Step 5: Evaluate
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2%}")
```

This is the same for EVERY model. Change the import, everything else stays the same:

```
# Decision Tree
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()

# Random Forest
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()

# SVM
from sklearn.svm import SVC
model = SVC()

# KNN
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier()

# ALL use: model.fit(X_train, y_train) → model.predict(X_test)
```

2. Key Functions

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Split data: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Scale features (important for many models)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # Use same scaler!

# Train
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

# Predict and evaluate
predictions = model.predict(X_test_scaled)
print(f"Accuracy: {accuracy_score(y_test, predictions):.2%}")
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

3. A Complete ExamGuard Example

Let's say you have extracted FEATURES from video clips (before using deep learning):

```
"""
ExamGuard: Simple Cheating Detection with Scikit-Learn
Features extracted from video clips:
  - avg_gaze_deviation: How much the student looks away from their paper
  - head_movement_freq: How often they move their head
  - hand_movement_range: How far their hands move from the desk
  - stillness_duration: Longest period of no movement
  - neighbor_proximity_score: How close to their neighbor's space
"""
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Simulated feature data
np.random.seed(42)
n_normal = 900
n_cheating = 100

# Normal students: low gaze deviation, low head movement
normal = np.column_stack([
    np.random.normal(10, 5, n_normal),   # gaze_deviation
    np.random.normal(3, 2, n_normal),    # head_movement
    np.random.normal(5, 3, n_normal),    # hand_movement
    np.random.normal(30, 10, n_normal),  # stillness
    np.random.normal(2, 1, n_normal),    # neighbor_proximity
])

# Cheating students: high gaze deviation, high head movement
cheating = np.column_stack([
    np.random.normal(35, 10, n_cheating),  # gaze_deviation
    np.random.normal(15, 5, n_cheating),   # head_movement
    np.random.normal(15, 5, n_cheating),   # hand_movement
```

```
    np.random.normal(10, 5, n_cheating),   # stillness
    np.random.normal(7, 2, n_cheating),    # neighbor_proximity
])

# Combine
X = np.vstack([normal, cheating])
y = np.array([0] * n_normal + [1] * n_cheating)  # 0=normal, 1=cheating

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
predictions = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, predictions):.2%}")
print("\nDetailed Report:")
print(classification_report(y_test, predictions,
                            target_names=["Normal", "Cheating"]))

# Feature importance (which features matter most?)
feature_names = ["gaze_dev", "head_move", "hand_move", "stillness", "neighbor"]
importances = model.feature_importances_
for name, imp in sorted(zip(feature_names, importances), key=lambda x: -x[1]):
    print(f"  {name}: {imp:.3f}")
```

ExamGuard connection: This shows the COMPLETE ML workflow that you'll use for every model, just with more complex features and models later.

Mini Project: Build a Simple Cheating Classifier

```
"""
Mini Project: ExamGuard Simple Classifier
Try 4 different models and compare them.
"""
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Generate data (same as above)
np.random.seed(42)
normal = np.column_stack([
    np.random.normal(10, 5, 900),
```

```
    np.random.normal(3, 2, 900),
    np.random.normal(5, 3, 900),
])
cheating = np.column_stack([
    np.random.normal(30, 10, 100),
    np.random.normal(12, 5, 100),
    np.random.normal(14, 5, 100),
])

X = np.vstack([normal, cheating])
y = np.array([0] * 900 + [1] * 100)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Try 4 models
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "SVM": SVC(random_state=42),
    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5),
}

print("=== ExamGuard Model Comparison ===\n")
print(f"{'Model':<20} {'Accuracy':>10} {'F1 Score':>10}")
print("-" * 42)

for name, model in models.items():
    model.fit(X_train_s, y_train)
    preds = model.predict(X_test_s)
    acc = accuracy_score(y_test, preds)
    f1 = f1_score(y_test, preds)
    print(f"{name:<20} {acc:>10.2%} {f1:>10.3f}")

print("\nBest model for ExamGuard: Pick highest F1 score")
print("(F1 matters more than accuracy for imbalanced data!)")
```

Key Scikit-Learn Functions

```
# Data Splitting
train_test_split(X, y, test_size=0.2, stratify=y)

# Preprocessing
StandardScaler()          # Normalize features
LabelEncoder()            # Convert text labels to numbers

# Models
```

```
DecisionTreeClassifier()          # Simple, interpretable
RandomForestClassifier()          # Powerful, handles many features
SVC()                             # Support Vector Machine
KNeighborsClassifier()            # Instance-based learning

# Training and Prediction
model.fit(X_train, y_train)       # Train
model.predict(X_test)             # Predict
model.predict_proba(X_test)       # Probability scores

# Evaluation
accuracy_score(y_true, y_pred)    # Overall accuracy
f1_score(y_true, y_pred)          # Balance of precision/recall
classification_report()           # Complete report
confusion_matrix()                # Detailed breakdown
```

From Scikit-Learn to Deep Learning

```
Scikit-Learn:
  model = RandomForestClassifier()
  model.fit(X_train, y_train)
  preds = model.predict(X_test)

PyTorch (Deep Learning):
  model = CNN()                   # Different model definition
  for epoch in range(100):        # Training loop (more control)
      output = model(X_train)
      loss = criterion(output, y_train)
      loss.backward()
      optimizer.step()
  preds = model(X_test)
```

The WORKFLOW is the same: prepare data → train → predict → evaluate.
Deep learning just gives you more control over the training process.

Scikit-Learn is where you learn the workflow. Deep learning is where you apply it to complex problems like computer vision.

Data Preprocessing

What is Data Preprocessing?

Data preprocessing is cleaning and preparing your data before feeding it to a machine learning model. Raw data is messy, inconsistent, and not in the format models expect.

Think of it like preparing ingredients before cooking:

- You don't put a whole chicken in the pot (you need to clean, cut, and season it)
- You don't feed raw, messy data to a model (you need to clean, transform, and format it)

Why Data Preprocessing Matters for ExamGuard

The Golden Rule: Garbage In = Garbage Out

If you feed bad data to the best model in the world, you get bad results. Period.

```
RAW exam data problems:
 - Video frames are different sizes (Camera 1: 1920x1080, Camera 2: 1280x720)
 - Some clips are too dark (night exam) or too bright (window glare)
 - Labels might be inconsistent ("phone", "Phone", "mobile", "cell phone")
 - Some clips are corrupted or too short
 - Pixel values range 0-255 but models expect 0-1
 - Missing metadata (no camera ID, no timestamp)

ALL of these must be fixed BEFORE training.
```

What happens without preprocessing:

```
Without preprocessing:
 Raw data → Model → 45% accuracy → Useless

With preprocessing:
 Raw data → Clean → Normalize → Format → Model → 92% accuracy → Useful!
```

What to Learn

1. Image Resizing (All Images MUST Be the Same Size)

```
import cv2
import numpy as np

# Problem: YOLO needs 640x640, camera gives 1920x1080
frame = cv2.imread("exam_frame.jpg")  # Shape: (1080, 1920, 3)

# Solution: Resize
resized = cv2.resize(frame, (640, 640))  # Shape: (640, 640, 3)
```

Why: Neural networks have a fixed input size. You can't feed a 1920x1080 image to a model expecting 640x640. EVERY image must be resized to the SAME dimensions.

ExamGuard specifics:

Model	Required Input Size	Why This Size
YOLOv8	640 x 640	Standard YOLO input

CNN (gaze)	224 x 224	Standard CNN input (ImageNet)
Face crop	112 x 112	Standard for face models
Behavior LSTM	224 x 224 per frame	Matches CNN input

2. Normalization (Scale Numbers to 0-1)

```
import numpy as np

# Problem: Pixel values are 0-255
frame = np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8)
print(f"Before: min={frame.min()}, max={frame.max()}")  # 0 to 255

# Solution: Normalize to 0-1
normalized = frame.astype(np.float32) / 255.0
print(f"After: min={normalized.min():.2f}, max={normalized.max():.2f}")  # 0.0 to 1.0
```

Why: Neural networks learn better when input values are small (0-1) instead of large (0-255). Large values cause unstable training (gradients explode).

Common normalization methods:

```
# Method 1: Simple division (0-1 range)
normalized = image / 255.0

# Method 2: ImageNet standardization (used with pre-trained models)
mean = np.array([0.485, 0.456, 0.406])  # ImageNet mean
std = np.array([0.229, 0.224, 0.225])   # ImageNet std
standardized = (image / 255.0 - mean) / std

# Method 3: Min-Max scaling for numerical features
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
features_scaled = scaler.fit_transform(features)
```

3. Handling Missing Data

```
import pandas as pd

# Load clip metadata
df = pd.read_csv("exam_clips.csv")

# Check for missing values
print(df.isnull().sum())
# clip_id          0
# label            5  ← 5 clips without labels
# duration_sec     2  ← 2 clips with missing duration
# camera_id        0
# confidence      15  ← 15 clips missing confidence

# Strategy 1: Drop rows with missing labels (can't train without labels!)
df = df.dropna(subset=["label"])

# Strategy 2: Fill missing numerical values with the median
df["duration_sec"] = df["duration_sec"].fillna(df["duration_sec"].median())
df["confidence"] = df["confidence"].fillna(df["confidence"].median())
```

```
print(f"Clean dataset: {len(df)} clips")
print(f"Missing values: {df.isnull().sum().sum()}")  # Should be 0
```

ExamGuard connection: Real datasets ALWAYS have missing data. Some clips might not have labels, some might have corrupted metadata. Handle this BEFORE training.

4. Label Encoding (Convert Text to Numbers)

```
# Problem: Models need numbers, not text
labels = ["normal", "phone", "looking_neighbor", "passing_notes"]

# Solution 1: Label encoding (for simple classification)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
encoded = le.fit_transform(labels)
print(encoded)  # [2, 3, 1, 0]  (alphabetical order)

# Solution 2: One-hot encoding (for neural networks)
# "phone"          → [0, 0, 1, 0]
# "normal"         → [0, 1, 0, 0]
# "looking_neighbor"→ [1, 0, 0, 0]
# "passing_notes"   → [0, 0, 0, 1]
```

ExamGuard connection: The model can't understand "phone" or "normal". It only understands numbers. Every label must be encoded.

5. Data Augmentation (Create More Training Data)

```
import cv2
import numpy as np

# Problem: Only 200 cheating clips (not enough!)
# Solution: Create variations of existing clips

def augment_frame(frame):
    """Create augmented versions of a frame."""
    augmented = []

    # Horizontal flip
    augmented.append(cv2.flip(frame, 1))

    # Brightness adjustment
    bright = cv2.convertScaleAbs(frame, alpha=1.2, beta=30)
    augmented.append(bright)

    # Slight rotation
    h, w = frame.shape[:2]
    M = cv2.getRotationMatrix2D((w//2, h//2), 5, 1.0)  # 5 degree rotation
    rotated = cv2.warpAffine(frame, M, (w, h))
    augmented.append(rotated)

    # Add slight noise
    noise = np.random.normal(0, 10, frame.shape).astype(np.uint8)
```

```
    noisy = cv2.add(frame, noise)
    augmented.append(noisy)

    return augmented

# 200 clips × 5 variations (original + 4 augmented) = 1000 clips!
```

ExamGuard connection: You have very few cheating clips. Augmentation creates more training data from existing clips by applying transformations. This is ESSENTIAL for handling the imbalanced data problem.

6. Video-Specific Preprocessing

```
import cv2

def preprocess_video_clip(video_path, target_frames=30, target_size=(224, 224)):
    """
    Preprocess a video clip for the behavior model.

    Steps:
    1. Load video
    2. Sample fixed number of frames
    3. Resize each frame
    4. Normalize pixel values
    """
    cap = cv2.VideoCapture(video_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Sample evenly spaced frames
    indices = np.linspace(0, total_frames - 1, target_frames, dtype=int)

    frames = []
    for idx in indices:
        cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
        ret, frame = cap.read()
        if ret:
            # Resize
            frame = cv2.resize(frame, target_size)
            # Convert BGR to RGB (OpenCV uses BGR, models expect RGB)
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            # Normalize
            frame = frame.astype(np.float32) / 255.0
            frames.append(frame)

    cap.release()

    # Stack into array: (30, 224, 224, 3)
    return np.array(frames)
```

ExamGuard connection: Video clips have different lengths (20 sec, 45 sec, etc.). The LSTM model needs a FIXED number of frames. This function standardizes every clip to exactly 30 frames.

The Complete Preprocessing Pipeline for ExamGuard

```
RAW DATA
   |
   v
1. COLLECT: Load all video clips and metadata CSV
   |
   v
2. CLEAN: Remove corrupted clips, fix missing labels, standardize label names
   |
   v
3. RESIZE: All frames to required size (640x640 for YOLO, 224x224 for CNN)
   |
   v
4. NORMALIZE: Pixel values from 0-255 to 0.0-1.0
   |
   v
5. AUGMENT: Create more cheating clips via flip, rotate, brightness
   |
   v
6. ENCODE: Convert text labels to numbers
   |
   v
7. SPLIT: 80% training, 20% testing (stratified!)
   |
   v
8. SAVE: Store preprocessed data for fast loading during training
   |
   v
CLEAN DATA → Ready for model training!
```

Mini Project: Build a Preprocessing Pipeline

```
"""
Mini Project: ExamGuard Data Preprocessing Pipeline
Practice: Resizing, normalizing, augmenting, encoding
"""
import numpy as np

# Simulate raw data
print("=== ExamGuard Preprocessing Pipeline ===\n")

# Step 1: Simulate raw frames of different sizes
raw_frames = [
    np.random.randint(0, 255, (1080, 1920, 3)),  # Camera 1: Full HD
    np.random.randint(0, 255, (720, 1280, 3)),   # Camera 2: HD
    np.random.randint(0, 255, (480, 640, 3)),    # Camera 3: SD
]

print("Step 1 - Raw frames:")
for i, frame in enumerate(raw_frames):
    print(f"  Camera {i+1}: {frame.shape}")
```

```
# Step 2: Resize all to same size
target_size = (224, 224)
resized = []
for frame in raw_frames:
    # Simple resize by taking evenly spaced pixels (real code uses cv2.resize)
    h_idx = np.linspace(0, frame.shape[0]-1, target_size[0], dtype=int)
    w_idx = np.linspace(0, frame.shape[1]-1, target_size[1], dtype=int)
    r = frame[np.ix_(h_idx, w_idx)]
    resized.append(r)

print("\nStep 2 - After resizing:")
for i, frame in enumerate(resized):
    print(f"  Camera {i+1}: {frame.shape}")

# Step 3: Normalize
normalized = [f.astype(np.float32) / 255.0 for f in resized]
print(f"\nStep 3 - After normalizing:")
print(f"  Value range: {normalized[0].min():.2f} to {normalized[0].max():.2f}")

# Step 4: Label encoding
raw_labels = ["normal", "phone", "looking_neighbor", "normal", "passing_notes"]
label_map = {"normal": 0, "phone": 1, "looking_neighbor": 2, "passing_notes": 3}
encoded_labels = [label_map[l] for l in raw_labels]
print(f"\nStep 4 - Label encoding:")
for raw, enc in zip(raw_labels, encoded_labels):
    print(f"  '{raw}' → {enc}")

# Step 5: Data augmentation (simulate)
n_original_cheating = 50
n_augmented_per_clip = 4
n_total = n_original_cheating * (1 + n_augmented_per_clip)
print(f"\nStep 5 - Data augmentation:")
print(f"  Original cheating clips: {n_original_cheating}")
print(f"  Augmentations per clip: {n_augmented_per_clip}")
print(f"  Total after augmentation: {n_total}")

# Step 6: Summary
print(f"\n=== Pipeline Complete ===")
print(f"  All frames: {target_size[0]}x{target_size[1]} pixels")
print(f"  Value range: 0.0 to 1.0")
print(f"  Labels: numerical (0-3)")
print(f"  Data: augmented and balanced")
print(f"  Ready for training!")
```

Common Preprocessing Mistakes

Mistake	Why It's Bad	How to Avoid
Not resizing images	Model crashes or performs poorly	Always resize to model's expected input
Forgetting to normalize	Training is unstable, slow	Always normalize to 0-1 or standardize

Normalizing test data with test statistics	Data leakage!	Fit scaler on TRAIN data only, apply to test
Inconsistent labels	Model gets confused	Standardize labels before training
Not augmenting minority class	Model ignores rare events	Augment cheating clips heavily
BGR vs RGB mix-up	Colors are wrong, model confused	OpenCV reads BGR, convert to RGB for models
Preprocessing after split	Data leakage risk	Determine preprocessing params from train set only

The most dangerous mistake is data leakage: accidentally using information from the test set during training. This makes your metrics look great but the model fails in the real world.

Train-Test Split

What is Train-Test Split?

Train-test split is dividing your data into two separate groups before training:

- Training set (80%): The model learns from this data
- Testing set (20%): The model is evaluated on this data (it NEVER sees this during training)

```
All Data (10,000 clips)
  |
  ├── Training Set (8,000 clips) → Model LEARNS from these
  |
  └── Testing Set (2,000 clips)  → Model is TESTED on these
```

Why Train-Test Split Matters for ExamGuard

The Exam Analogy

Imagine a student who memorized the exact questions from last year's exam paper:

```
Teacher gives them last year's paper as a test:
  → Student scores 100%
  → Teacher thinks: "Brilliant student!"

Teacher gives them a NEW paper:
  → Student scores 30%
  → Reality: Student memorized, didn't learn
```

This is EXACTLY what happens with ML models:

```
Model tested on training data:
  → 99% accuracy!
  → You think: "Amazing model!"

Model tested on NEW data (test set):
  → 55% accuracy
  → Reality: Model memorized, didn't learn

This is called OVERFITTING - the most common ML mistake.
```

For ExamGuard, this means:

If you test your phone detection model on the same frames it was trained on, it might score 99% accuracy. But in a REAL exam with NEW students, NEW angles, NEW lighting, it might only score 60%. That's useless.

The test set simulates the real world. If the model performs well on data it has NEVER seen, it will likely perform well in a real exam.

What to Learn

1. Basic Split

```
from sklearn.model_selection import train_test_split

# X = features (video clip data)
# y = labels (0=normal, 1=cheating)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% for testing
    random_state=42,    # Reproducible split
)

print(f"Training samples: {len(X_train)}")  # 8,000
print(f"Testing samples: {len(X_test)}")    # 2,000
```

2. Stratified Split (CRITICAL for ExamGuard!)

```
# Problem: Random split might put ALL cheating clips in one set

# Example of BAD random split:
# Training: 8,000 clips (7,950 normal, 50 cheating)   ← almost no cheating!
# Testing:  2,000 clips (1,550 normal, 450 cheating)  ← too much cheating!

# Solution: Stratified split preserves the ratio
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y          # ← THIS is the key parameter!
)

# Now both sets have the same ratio:
# Training: 8,000 clips (7,600 normal, 400 cheating)  = 5% cheating
# Testing:  2,000 clips (1,900 normal, 100 cheating)  = 5% cheating
```

Why stratified? ExamGuard has 95% normal and 5% cheating clips. A random split might accidentally put most cheating clips in one set. Stratified split ensures BOTH sets have the same proportion.

3. Validation Set (Three-Way Split)

For serious model development, you actually need THREE sets:

```
All Data (10,000 clips)
    |
    ├── Training Set (70%) - 7,000 clips → Model LEARNS
    |
    ├── Validation Set (15%) - 1,500 clips → Tune model settings
    |
    └── Test Set (15%) - 1,500 clips → FINAL evaluation (use only once!)
```

```
from sklearn.model_selection import train_test_split

# First split: separate test set
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
)

# Second split: separate validation from training
X_train, X_val, y_train, y_val = train_test_split(
```

```
    X_temp, y_temp, test_size=0.176, random_state=42, stratify=y_temp
)
# 0.176 of 85% ≈ 15% of total

print(f"Training:   {len(X_train)} clips ({len(X_train)/len(X)*100:.0f}%)")
print(f"Validation: {len(X_val)} clips ({len(X_val)/len(X)*100:.0f}%)")
print(f"Testing:    {len(X_test)} clips ({len(X_test)/len(X)*100:.0f}%)")
```

Why three sets?

Set	Purpose	When Used
Training	Model learns patterns	Every training epoch
Validation	Tune hyperparameters (learning rate, model size)	During development
Test	Final, unbiased evaluation	ONCE, at the very end

The validation set prevents you from accidentally tuning your model to the test set (which would be another form of cheating).

4. ExamGuard-Specific Split Strategy

For ExamGuard, a smarter split is by exam hall, not by random clips:

```
# BAD: Random split
# Training might have clips from Hall A, Row 3, Seat 7
# Testing might have OTHER clips from Hall A, Row 3, Seat 7
# The model might recognize the SEAT, not the BEHAVIOR!

# GOOD: Split by hall
# Training: Halls A, B, C (different students, different lighting)
# Testing:  Halls D, E (completely new environment)
# Now the model MUST generalize to new halls!

import pandas as pd

df = pd.read_csv("exam_clips.csv")

# Split by hall
train_halls = ["hall_A", "hall_B", "hall_C", "hall_D"]
test_halls = ["hall_E", "hall_F"]

train_data = df[df["hall_id"].isin(train_halls)]
test_data = df[df["hall_id"].isin(test_halls)]

print(f"Training: {len(train_data)} clips from halls {train_halls}")
print(f"Testing:  {len(test_data)} clips from halls {test_halls}")
```

This is harder to pass but gives you a model that actually works in the real world.

CRITICAL RULES (Memorize These!)

Rule 1: NEVER Test on Training Data

```
WRONG:
 model.fit(X, y)           # Train on ALL data
 predictions = model.predict(X)  # Test on SAME data
```

```
  accuracy = 99%              # FAKE accuracy!

RIGHT:
  model.fit(X_train, y_train)      # Train on training set
  predictions = model.predict(X_test) # Test on UNSEEN data
  accuracy = 85%                   # REAL accuracy!
```

Rule 2: NEVER Preprocess Using Test Data

```
WRONG:
  # Calculate mean from ALL data (includes test)
  mean = X.mean()
  X_normalized = X - mean  # Test data influenced the mean!

RIGHT:
  # Calculate mean from TRAINING data only
  mean = X_train.mean()
  X_train_normalized = X_train - mean  # Training normalized with training stats
  X_test_normalized = X_test - mean    # Test normalized with TRAINING stats
```

Rule 3: ALWAYS Use Stratified Split for Imbalanced Data

```
ExamGuard data: 95% normal, 5% cheating

Without stratify: Test set might have 0% cheating → can't test at all!
With stratify:    Test set has 5% cheating → proper testing
```

Rule 4: Split BEFORE Augmentation

```
WRONG:
  Augment data → Split → Training and test share augmented versions of same clips!

RIGHT:
  Split → Augment ONLY training set → Test set has ONLY original clips
```

If you augment before splitting, the test set might contain a flipped version of a training clip. The model would partially recognize it, giving fake high accuracy.

Mini Project: Demonstrate the Danger of Not Splitting

```
"""
Mini Project: Show WHY train-test split matters
Compare accuracy WITH and WITHOUT proper splitting
"""
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

np.random.seed(42)

# Generate data
n = 1000
X = np.random.randn(n, 5)
y = (X[:, 0] + X[:, 1] > 0).astype(int)
```

```
# Method 1: WRONG - test on training data
model_wrong = RandomForestClassifier(n_estimators=100, random_state=42)
model_wrong.fit(X, y)
preds_wrong = model_wrong.predict(X)
acc_wrong = accuracy_score(y, preds_wrong)

# Method 2: RIGHT - proper train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model_right = RandomForestClassifier(n_estimators=100, random_state=42)
model_right.fit(X_train, y_train)
preds_right = model_right.predict(X_test)
acc_right = accuracy_score(y_test, preds_right)

print("=== The Danger of Not Splitting ===\n")
print(f"Testing on training data (WRONG): {acc_wrong:.2%}")
print(f"Testing on unseen data (RIGHT):   {acc_right:.2%}")
print(f"\nDifference: {acc_wrong - acc_right:.2%}")
print(f"\nThe model MEMORIZED {acc_wrong - acc_right:.2%} of the training data.")
print(f"Only {acc_right:.2%} accuracy is real generalization.")
print(f"\nIf you deploy the model thinking it's {acc_wrong:.2%} accurate,")
print(f"it will actually perform at ~{acc_right:.2%} in the real exam hall.")
```

Summary

```
+--------------------------------------------+
|        TRAIN-TEST SPLIT CHECKLIST          |
+--------------------------------------------+
|                                            |
| [ ] Split BEFORE any preprocessing         |
| [ ] Use stratify=y for imbalanced data     |
| [ ] NEVER test on training data            |
| [ ] Augment ONLY the training set          |
| [ ] Consider splitting by hall/session     |
| [ ] Use validation set for tuning          |
| [ ] Use test set ONCE for final eval       |
| [ ] Typical split: 70/15/15 or 80/20       |
|                                            |
+--------------------------------------------+
```

Evaluation Metrics

What Are Evaluation Metrics?

Evaluation metrics are measurements that tell you how good your model actually is. They answer the question: "Is this model ready for the real world?"

You wouldn't buy a car just because the salesman says "it's great." You'd check the mileage, safety rating, and reliability scores. Same with ML models - you need objective measurements.

Why Evaluation Metrics Matter for ExamGuard

The Accuracy Trap

Here's a scenario that trips up every beginner:

ExamGuard dataset:
- 9,900 normal clips (99%)
- 100 cheating clips (1%)

You build a model. It predicts "NOT CHEATING" for EVERY SINGLE clip.

Accuracy = 9,900 correct / 10,000 total = 99% accuracy!

But the model caught ZERO cheating. It's completely useless.

99% accuracy. 0% useful. This is why accuracy alone is DANGEROUS for ExamGuard.

You need metrics that answer specific questions:

Question	Metric	ExamGuard Meaning
How often is the model correct overall?	Accuracy	Basic, but misleading for imbalanced data
Of all alerts sent, how many were real cheating?	Precision	Avoid bothering invigilators with false alarms
Of all real cheating, how much did we catch?	Recall	Don't miss actual cheating
What's the balance of precision and recall?	F1 Score	Overall model quality for ExamGuard

The Confusion Matrix: Foundation of All Metrics

Every prediction falls into one of four categories:

```
                ACTUAL
            Cheating    Normal
          +----------+----------+
Predicted |   TRUE   |  FALSE   |
Cheating  | POSITIVE | POSITIVE |
          |   (TP)   |   (FP)   |
          +----------+----------+
Predicted |  FALSE   |   TRUE   |
Normal    | NEGATIVE | NEGATIVE |
          |   (FN)   |   (TN)   |
          +----------+----------+
```

ExamGuard Examples:

True Positive (TP) - Correct Alert:

Model says: "Cheating!" → Student WAS actually cheating.

This is what we want!

False Positive (FP) - False Alarm:

Model says: "Cheating!" → Student was just scratching their head.

Invigilator walks over for nothing. Annoying but not dangerous.

True Negative (TN) - Correct Silence:

Model says: "Normal" → Student IS writing normally.

Good - no unnecessary alert.

False Negative (FN) - Missed Cheating:

Model says: "Normal" → Student WAS actually cheating with a phone!

THE WORST OUTCOME. We missed real cheating.

With Numbers:

ExamGuard tested on 2,000 clips (1,900 normal + 100 cheating)

```
                ACTUAL
            Cheating   Normal
          +----------+----------+
Predicted |    85    |    30    | → 115 alerts sent
Cheating  |  (TP)    |  (FP)    |
          +----------+----------+
Predicted |    15    |  1,870   | → 1,885 no alert
Normal    |  (FN)    |  (TN)    |
          +----------+----------+
            100        1,900
```

Results:

- 85 cheaters caught correctly (TP)
- 30 false alarms (FP) - students wrongly flagged
- 15 cheaters MISSED (FN) - real cheating went undetected
- 1,870 normal correctly ignored (TN)

The Metrics Explained

Accuracy

Accuracy = (TP + TN) / (TP + FP + FN + TN)
= (85 + 1,870) / 2,000
= 97.75%

Sounds great, but remember: a model that ALWAYS says "normal" would get 95% accuracy. So 97.75% is only slightly better than doing nothing.

ExamGuard verdict: Useful as a baseline, but NEVER rely on accuracy alone.

Precision

Precision = TP / (TP + FP)
= 85 / (85 + 30)
= 73.9%

"Of all alerts we sent, 73.9% were real cheating"
"26.1% of our alerts were false alarms"

ExamGuard meaning: If precision is too low, invigilators get flooded with false alarms and start ignoring ALL alerts. We want precision to be high (>80%).

Think of it as: "How much can the invigilator TRUST our alerts?"

Recall (Sensitivity)

Recall = TP / (TP + FN)
= 85 / (85 + 15)
= 85.0%

"We caught 85% of all real cheating"
"We MISSED 15% of real cheating"

ExamGuard meaning: If recall is too low, cheating goes undetected. We want recall to be very high (>90%).

Think of it as: "How much cheating are we actually catching?"

F1 Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.739 × 0.850) / (0.739 + 0.850)
= 79.0%

F1 is the harmonic mean of precision and recall.
It's only high when BOTH precision and recall are high.

ExamGuard meaning: F1 gives you ONE number that balances "don't cry wolf" (precision) with "don't miss anything" (recall).

Think of it as: "Overall, how good is our detection system?"

The Precision-Recall Tradeoff

You can't maximize both. Here's why:

HIGH THRESHOLD (e.g., alert only if >95% confident):
→ Very few alerts sent
→ Almost all alerts are real cheating (HIGH PRECISION)
→ But many cheating incidents below 95% are missed (LOW RECALL)

LOW THRESHOLD (e.g., alert if >30% confident):
→ Many alerts sent
→ Catches almost all cheating (HIGH RECALL)
→ But lots of false alarms (LOW PRECISION)

ExamGuard Threshold Examples:

Threshold	Alerts Sent	Precision	Recall	F1	Effect
0.30	500	20%	98%	33%	Catch

					everything but too many false alarms
0.50	200	45%	92%	60%	Still too many false alarms
0.70	115	74%	85%	79%	Good balance
0.80	80	88%	70%	78%	Fewer false alarms but missing cheating
0.95	30	97%	29%	45%	Almost no false alarms but missing most cheating

For ExamGuard, a threshold around 0.70-0.80 is usually best. You want to catch most cheating (recall > 80%) without overwhelming invigilators with false alarms (precision > 70%).

Which Metric Matters Most for ExamGuard?

Priority order for ExamGuard:

1. RECALL (most important)
   → Missing real cheating is the WORST outcome
   → Target: > 85%

2. PRECISION (second most important)
   → Too many false alarms = invigilators ignore the system
   → Target: > 70%

3. F1 SCORE (overall quality)
   → Balance of the above
   → Target: > 75%

4. ACCURACY (least useful for ExamGuard)
   → Misleading with imbalanced data
   → Don't optimize for this

Why recall matters more:

Scenario A: Miss 1 real cheater (low recall)
→ That student gets an unfair advantage
→ Other students suffer
→ System has failed its core purpose

Scenario B: 10 extra false alarms (low precision)
→ Invigilator checks 10 innocent students
→ Annoying but no one is harmed
→ System still works, just noisily

Missing cheating is WORSE than false alarms. So we prioritize recall, then optimize precision.

Implementing Evaluation in Code

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import numpy as np

# Simulated predictions
np.random.seed(42)
y_true = np.array([0]*1900 + [1]*100)  # Actual labels
y_pred = np.array([0]*1870 + [1]*30 + [1]*85 + [0]*15)  # Predictions

# All metrics at once
print("=== ExamGuard Model Evaluation ===\n")
print(f"Accuracy:  {accuracy_score(y_true, y_pred):.2%}")
print(f"Precision: {precision_score(y_true, y_pred):.2%}")
print(f"Recall:    {recall_score(y_true, y_pred):.2%}")
print(f"F1 Score:  {f1_score(y_true, y_pred):.2%}")

# Confusion Matrix
print(f"\nConfusion Matrix:")
cm = confusion_matrix(y_true, y_pred)
print(f"  TN={cm[0][0]}  FP={cm[0][1]}")
print(f"  FN={cm[1][0]}  TP={cm[1][1]}")

# Full report
print(f"\nFull Classification Report:")
print(classification_report(y_true, y_pred,
                            target_names=["Normal", "Cheating"]))
```

Mini Project: Evaluate Different Thresholds

```
"""
Mini Project: Find the Best Alert Threshold for ExamGuard
Practice: Precision, recall, F1, threshold selection
"""
import numpy as np

np.random.seed(42)

# Simulate model confidence scores for 2000 test clips
n_normal = 1900
n_cheating = 100

# Normal behavior: high confidence of being normal (low suspicion)
normal_scores = np.random.beta(2, 8, n_normal)  # Mostly low scores

# Cheating behavior: higher suspicion scores
cheating_scores = np.random.beta(5, 3, n_cheating)  # Mostly higher scores

all_scores = np.concatenate([normal_scores, cheating_scores])
all_labels = np.array([0]*n_normal + [1]*n_cheating)
```

```
# Try different thresholds
print("=== ExamGuard Threshold Analysis ===\n")
print(f"{'Threshold':>10} {'Alerts':>7} {'Precision':>10} {'Recall':>8} {'F1':>6}")
print("-" * 50)

best_f1 = 0
best_threshold = 0

for threshold in np.arange(0.1, 0.95, 0.05):
    predictions = (all_scores >= threshold).astype(int)

    tp = np.sum((predictions == 1) & (all_labels == 1))
    fp = np.sum((predictions == 1) & (all_labels == 0))
    fn = np.sum((predictions == 0) & (all_labels == 1))

    alerts = tp + fp
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    marker = ""
    if f1 > best_f1:
        best_f1 = f1
        best_threshold = threshold
        marker = " <-- best F1"

    print(f"{threshold:>10.2f} {alerts:>7} {precision:>10.1%} {recall:>8.1%} {f1:>6.1%}{marker}")

print(f"\nBest threshold: {best_threshold:.2f} (F1 = {best_f1:.1%})")
print(f"\nRecommendation: Use threshold {best_threshold:.2f} for ExamGuard")
```

Key Takeaways

```
+-------------------------------------------------------+
|  EXAMGUARD EVALUATION RULES                           |
+-------------------------------------------------------+
|                                                       |
|  1. NEVER rely on accuracy alone (99% can be useless)|
|  2. Precision = "Can invigilators trust alerts?"     |
|  3. Recall = "Are we catching enough cheating?"      |
|  4. F1 = "Overall system quality"                    |
|  5. Recall > Precision for ExamGuard (missing        |
|     cheating is worse than false alarms)             |
|  6. Always look at the confusion matrix              |
|  7. Try different thresholds to find the sweet spot  |
|                                                       |
+-------------------------------------------------------+
```

Imbalanced Data

What is Imbalanced Data?

Imbalanced data is when one class has WAY more examples than another. The common class is called the "majority class" and the rare class is called the "minority class."

```
Balanced data:     50% Class A, 50% Class B     (easy to learn)
Imbalanced data:   99% Class A, 1% Class B      (VERY hard to learn)
```

Why Imbalanced Data is ExamGuard's Biggest Challenge

The Reality of Exam Monitoring

In a real exam:

- 99% of the time, students are writing normally
- Only about 1% of behavior involves cheating

This means your training data looks like:

```
ExamGuard Training Data:
  Normal behavior clips:   9,900 (99%)
  Cheating clips:          100 (1%)
  ──────────────────────────────────
  Total:                   10,000

Visualized:
  Normal:
████████████████████████████████████████████████████████████████████████████████
99%
  Cheating: ▌ 1%
```

What Happens If You Ignore This?

The model takes the lazy shortcut:

```
Model's "learning":
  "If I ALWAYS predict 'normal', I'm right 99% of the time!"

Result:
  Accuracy: 99%
  Cheating caught: 0 out of 100
  The model is USELESS. It never learned what cheating looks like.
```

This is not a theoretical problem. This WILL happen if you don't handle imbalanced data. It's one of the most common reasons real ML projects fail.

Why Models Fail with Imbalanced Data

The Model's Perspective

During training, the model sees examples and adjusts its weights:

```
Batch 1:  normal, normal, normal, normal, normal  → Learn "normal"
Batch 2:  normal, normal, normal, normal, normal  → Reinforce "normal"
Batch 3:  normal, normal, normal, normal, normal  → Even more "normal"
```

...
Batch 99: normal, normal, normal, normal, CHEATING → One cheating example!
But the model has seen 495 normal vs 1 cheating.
The cheating signal is drowned out.

The model simply doesn't see ENOUGH cheating examples to learn the pattern. It's like trying to learn Urdu by reading 99 English books and 1 Urdu book.

Solutions

Solution 1: Oversampling (Duplicate Minority Class)

Simply copy cheating clips multiple times to balance the dataset.

```
from sklearn.utils import resample
import pandas as pd

# Original data
df = pd.DataFrame({
   "clip_id": range(1000),
   "label": ["normal"] * 950 + ["cheating"] * 50
})

print(f"Before: {df['label'].value_counts().to_dict()}")
# normal: 950, cheating: 50

# Separate classes
normal = df[df["label"] == "normal"]
cheating = df[df["label"] == "cheating"]

# Oversample cheating to match normal
cheating_oversampled = resample(
   cheating,
   replace=True,        # Allow duplicates
   n_samples=len(normal), # Match majority class size
   random_state=42
)

# Combine
df_balanced = pd.concat([normal, cheating_oversampled])
print(f"After:  {df_balanced['label'].value_counts().to_dict()}")
# normal: 950, cheating: 950
```

Pros: Simple, works immediately

Cons: Exact duplicates can cause overfitting (model memorizes specific clips)

Solution 2: Undersampling (Reduce Majority Class)

Remove some normal clips to balance the dataset.

```
# Undersample normal to match cheating
normal_undersampled = resample(
   normal,
   replace=False,          # No duplicates
```

```
    n_samples=len(cheating),   # Match minority class size
    random_state=42
)

df_balanced = pd.concat([normal_undersampled, cheating])
print(f"After: {df_balanced['label'].value_counts().to_dict()}")
# normal: 50, cheating: 50
```

Pros: No duplicates, fast training (smaller dataset)

Cons: Throws away 900 normal clips! Loses valuable training data.

Solution 3: SMOTE (Create Synthetic Minority Examples)

SMOTE (Synthetic Minority Over-sampling Technique) creates NEW, artificial cheating examples by interpolating between existing ones.

```
from imblearn.over_sampling import SMOTE
import numpy as np

# Features and labels
X = np.random.randn(1000, 5)  # 5 features
y = np.array([0] * 950 + [1] * 50)  # 0=normal, 1=cheating

print(f"Before SMOTE: Normal={sum(y==0)}, Cheating={sum(y==1)}")

# Apply SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

print(f"After SMOTE:  Normal={sum(y_resampled==0)}, Cheating={sum(y_resampled==1)}")
# Before: Normal=950, Cheating=50
# After:  Normal=950, Cheating=950
```

How SMOTE works:

```
Original cheating clip features: [0.8, 0.7, 0.9]
Another cheating clip features:  [0.6, 0.5, 0.8]

SMOTE creates a new synthetic example BETWEEN them:
New synthetic features:         [0.7, 0.6, 0.85]

This is NOT a copy - it's a new, realistic example!
```

Pros: Creates diverse, new examples (better than plain duplication)

Cons: Works on features, not raw images. For images, use augmentation instead.

Solution 4: Data Augmentation (Best for Images/Video!)

For ExamGuard, data augmentation is the BEST solution because we work with images and video.

```
import cv2
import numpy as np

def augment_cheating_clip(frames):
    """Create 5 new variations of a cheating video clip."""
```

```
    augmented = []

    # Original
    augmented.append(frames)

    # Horizontal flip (mirror image)
    augmented.append([cv2.flip(f, 1) for f in frames])

    # Brightness increase (+20%)
    augmented.append([cv2.convertScaleAbs(f, alpha=1.2, beta=0) for f in frames])

    # Brightness decrease (-20%)
    augmented.append([cv2.convertScaleAbs(f, alpha=0.8, beta=0) for f in frames])

    # Add random noise
    noisy = []
    for f in frames:
        noise = np.random.normal(0, 10, f.shape).astype(np.uint8)
        noisy.append(cv2.add(f, noise))
    augmented.append(noisy)

    # Slight zoom (crop center 90% and resize back)
    zoomed = []
    for f in frames:
        h, w = f.shape[:2]
        crop = f[h//20:h-h//20, w//20:w-w//20]
        zoomed.append(cv2.resize(crop, (w, h)))
    augmented.append(zoomed)

    return augmented  # 6 versions (1 original + 5 augmented)

# Result:
# 100 original cheating clips × 6 = 600 cheating clips
# Much closer to the 950 normal clips!
```

Pros: Creates visually different examples, works great for images/video

Cons: All variations are based on original clips (limited diversity)

Solution 5: Class Weights (Tell the Model to Care More About Minority)

Instead of changing the data, tell the model to pay MORE attention to cheating examples.

```
from sklearn.ensemble import RandomForestClassifier

# Without class weights (model treats all samples equally)
model_bad = RandomForestClassifier()
model_bad.fit(X_train, y_train)

# With class weights (model pays more attention to cheating)
model_good = RandomForestClassifier(class_weight="balanced")
model_good.fit(X_train, y_train)

# "balanced" automatically calculates weights:
```

```
# Normal weight:  1000 / (2 * 950) = 0.526
# Cheating weight: 1000 / (2 * 50)  = 10.0
# Cheating examples count 19x more than normal!
```

For deep learning (PyTorch):

```
import torch
import torch.nn as nn

# Calculate weights inversely proportional to class frequency
n_normal = 9500
n_cheating = 500
total = n_normal + n_cheating

weight_normal = total / (2 * n_normal)    # 0.526
weight_cheating = total / (2 * n_cheating) # 10.0

class_weights = torch.tensor([weight_normal, weight_cheating])
criterion = nn.CrossEntropyLoss(weight=class_weights)

# Now the loss function penalizes missing cheating 19x more than missing normal
```

Pros: No data modification needed, easy to implement

Cons: Might make model too aggressive (more false alarms)

Which Solution to Use for ExamGuard?

Recommended combination:

```
1. Data Augmentation (PRIMARY)
  → Create more cheating clips via flip, rotate, brightness, noise
  → Goal: At least 5x more cheating clips

2. Class Weights (SECONDARY)
  → Tell the model cheating matters more
  → Helps even after augmentation

3. Stratified Sampling (ALWAYS)
  → Ensure every training batch has some cheating clips
  → Never let the model go many batches without seeing cheating

DO NOT use undersampling (you need all your normal data too)
Use SMOTE only for tabular features, not for raw images
```

Mini Project: Compare Imbalanced vs Balanced Training

```
"""
Mini Project: See the Impact of Imbalanced Data
Train the same model WITH and WITHOUT handling imbalance
"""
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

```
from sklearn.utils import resample

np.random.seed(42)

# Create imbalanced data
n_normal = 950
n_cheating = 50

normal_X = np.random.normal(0, 1, (n_normal, 5))
cheating_X = np.random.normal(2, 1, (n_cheating, 5))

X = np.vstack([normal_X, cheating_X])
y = np.array([0] * n_normal + [1] * n_cheating)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Method 1: Ignore imbalance (naive approach)
print("=== Method 1: Ignore Imbalance (NAIVE) ===")
model_naive = RandomForestClassifier(random_state=42)
model_naive.fit(X_train, y_train)
preds_naive = model_naive.predict(X_test)
print(classification_report(y_test, preds_naive, target_names=["Normal", "Cheating"]))

# Method 2: Class weights
print("=== Method 2: Class Weights ===")
model_weighted = RandomForestClassifier(class_weight="balanced", random_state=42)
model_weighted.fit(X_train, y_train)
preds_weighted = model_weighted.predict(X_test)
print(classification_report(y_test, preds_weighted, target_names=["Normal", "Cheating"]))

# Method 3: Oversampling
print("=== Method 3: Oversampling ===")
train_df_X = X_train
train_df_y = y_train

# Separate and oversample
cheating_mask = train_df_y == 1
normal_mask = train_df_y == 0

X_cheating = train_df_X[cheating_mask]
y_cheating = train_df_y[cheating_mask]

# Oversample cheating to match normal
n_to_sample = normal_mask.sum()
indices = np.random.choice(len(X_cheating), size=n_to_sample, replace=True)
X_cheating_over = X_cheating[indices]
y_cheating_over = y_cheating[indices]

X_balanced = np.vstack([train_df_X[normal_mask], X_cheating_over])
```

```
y_balanced = np.concatenate([train_df_y[normal_mask], y_cheating_over])

model_over = RandomForestClassifier(random_state=42)
model_over.fit(X_balanced, y_balanced)
preds_over = model_over.predict(X_test)
print(classification_report(y_test, preds_over, target_names=["Normal", "Cheating"]))

# Summary
from sklearn.metrics import recall_score, f1_score
print("=== Summary ===")
print(f"{'Method':<25} {'Recall (Cheating)':>20} {'F1 (Cheating)':>15}")
print("-" * 62)
print(f"{'Naive (no handling)':<25} {recall_score(y_test, preds_naive):>20.1%} {f1_score(y_test, preds_naive):>15.1%}")
print(f"{'Class Weights':<25} {recall_score(y_test, preds_weighted):>20.1%} {f1_score(y_test,
preds_weighted):>15.1%}")
print(f"{'Oversampling':<25} {recall_score(y_test, preds_over):>20.1%} {f1_score(y_test, preds_over):>15.1%}")
```

This is One of the HARDEST Problems in Real ML

Imbalanced data is not just an ExamGuard problem. It appears everywhere in the real world:

Domain	Majority Class	Minority Class	Ratio
ExamGuard	Normal behavior	Cheating	99:1
Fraud detection	Legitimate transactions	Fraud	99.9:0.1
Medical diagnosis	Healthy patients	Disease	95:5
Spam detection	Real email	Spam	80:20
Manufacturing	Good products	Defective	99:1

Every ML engineer struggles with this. The solutions you learn here apply to ALL of these domains.

Key Takeaways

```
+-------------------------------------------------------+
| IMBALANCED DATA SURVIVAL GUIDE                        |
+-------------------------------------------------------+
|                                                       |
| 1. CHECK your class distribution FIRST                |
|    (before any training!)                             |
|                                                       |
| 2. NEVER trust accuracy on imbalanced data            |
|    (use F1, precision, recall instead)                |
|                                                       |
| 3. For ExamGuard images/video:                        |
|    → Data augmentation (flip, rotate, brightness)     |
|    → Class weights in loss function                   |
|    → Stratified batches during training               |
|                                                       |
| 4. For tabular features:                              |
|    → SMOTE                                            |
|    → Class weights                                    |
|    → Oversampling                                     |
|                                                       |
| 5. ALWAYS use stratified train-test split             |
|                                                       |
```

| 6. Monitor recall for the minority class |
| (catching cheating is our #1 priority) |
| |
+---+

Phase 3: Core ML - Resources

Scikit-Learn

YouTube

- "Scikit-Learn Tutorial" by freeCodeCamp
- Complete walkthrough of the library
- Search: "freeCodeCamp scikit-learn tutorial"
- "Machine Learning with Scikit-Learn" by Data School
- Excellent playlist, very clear explanations
- Search: "Data School scikit-learn tutorial"
- "Scikit-Learn Crash Course" by Tech With Tim
- Quick but thorough overview
- Search: "Tech With Tim scikit-learn crash course"

Documentation

- Scikit-Learn Official Docs: scikit-learn.org/stable/
- The best reference once you understand the basics
- Especially: User Guide section

Practice

- Kaggle: Titanic Competition (kaggle.com/c/titanic)
- The classic beginner ML competition
- Practice the full workflow: load data, preprocess, train, evaluate
- Thousands of notebooks to learn from

Evaluation Metrics

YouTube

- "Precision, Recall, F1 Score" by StatQuest (Josh Starmer)
- THE BEST explanation of these metrics, period
- Search: "StatQuest precision recall"
- Also watch his confusion matrix video
- "ROC and AUC Explained" by StatQuest
- Another key metric, explained perfectly
- Search: "StatQuest ROC AUC"
- "Confusion Matrix Explained Simply" by Normalized Nerd
- Great visual explanation
- Search: "Normalized Nerd confusion matrix"
- "Accuracy is NOT Enough" by Krish Naik
- Why accuracy fails on imbalanced data
- Search: "Krish Naik accuracy imbalanced"

Must-Watch Order:

1. StatQuest: Confusion Matrix
2. StatQuest: Sensitivity and Specificity
3. StatQuest: Precision, Recall, F1
4. StatQuest: ROC and AUC

These four videos will give you a complete understanding of evaluation metrics.

Data Preprocessing

YouTube

- "Data Preprocessing in Machine Learning" by Krish Naik
- Covers all preprocessing steps
- Search: "Krish Naik data preprocessing ML"
- "Feature Scaling: Normalization vs Standardization" by StatQuest
- When to use which scaling method
- Search: "StatQuest feature scaling"
- "Data Augmentation for Deep Learning" by Aladdin Persson
- How to create more training data from existing images
- Search: "Aladdin Persson data augmentation"

Imbalanced Data

YouTube

- "Dealing with Imbalanced Data" by Krish Naik
- Covers all solutions (SMOTE, oversampling, class weights)
- Search: "Krish Naik imbalanced data"
- "SMOTE Explained" by Normalized Nerd
- Visual explanation of how SMOTE creates synthetic examples
- Search: "Normalized Nerd SMOTE"
- "Class Imbalance in Machine Learning" by Abhishek Thakur
- Practical tips from a Kaggle Grandmaster
- Search: "Abhishek Thakur class imbalance"

Library

- imbalanced-learn (imbalanced-learn.org)
- Python library specifically for handling imbalanced data
- Includes SMOTE, ADASYN, and other methods
- `pip install imbalanced-learn`

Train-Test Split

YouTube

- "Train Test Split Explained" by StatQuest
- Clear, visual explanation

- Search: "StatQuest train test split"
- "Cross Validation Explained" by StatQuest
- Advanced technique for better evaluation
- Search: "StatQuest cross validation"
- "Overfitting vs Underfitting" by StatQuest
- Why we split data in the first place
- Search: "StatQuest overfitting underfitting"

Courses (Comprehensive)

Free

- Andrew Ng's Machine Learning Course (Coursera)
- The most famous ML course in the world
- Covers all core ML concepts
- Free to audit (no certificate without paying)
- Search: "Andrew Ng machine learning Coursera"
- Google's Machine Learning Crash Course (developers.google.com/machine-learning)
- Free, interactive, by Google
- Covers all topics in this phase
- fast.ai Practical Deep Learning (course.fast.ai)
- Free, practical, top-down approach
- You'll use this more in later phases

Paid (Optional)

- "Hands-On Machine Learning" by Aurelien Geron (book)
- The best ML textbook, very practical
- Uses Scikit-Learn and TensorFlow
- Available as O'Reilly book

Practice Platforms

Kaggle Competitions (Start Here)

- Titanic (kaggle.com/c/titanic)
- Binary classification, imbalanced data
- Perfect practice for ExamGuard concepts
- Digit Recognizer (kaggle.com/c/digit-recognizer)
- Image classification (handwritten digits)
- First step toward computer vision
- Spam Detection datasets
- Text classification with imbalanced data
- Search "spam classification dataset" on Kaggle

Kaggle Learn Courses (Free)

- Intro to Machine Learning (kaggle.com/learn/intro-to-machine-learning)
- Intermediate Machine Learning (kaggle.com/learn/intermediate-machine-learning)
- These two cover exactly what's in this phase

Recommended Learning Order

Week 1: Scikit-Learn Basics
- Watch: freeCodeCamp Scikit-Learn tutorial
- Practice: Kaggle Titanic competition
- Goal: Complete the fit → predict → evaluate workflow

Week 2: Evaluation Metrics
- Watch: ALL StatQuest videos (confusion matrix, precision, recall, F1, ROC)
- Practice: Evaluate your Titanic model with all metrics
- Goal: Understand WHY accuracy alone is not enough

Week 3: Data Preprocessing + Train-Test Split
- Watch: Krish Naik data preprocessing
- Watch: StatQuest overfitting + train-test split
- Practice: Build a complete preprocessing pipeline
- Goal: Clean data, split properly, preprocess correctly

Week 4: Imbalanced Data
- Watch: Krish Naik imbalanced data + Normalized Nerd SMOTE
- Practice: Take an imbalanced dataset, apply all solutions, compare results
- Goal: Handle imbalanced data confidently (this is critical for ExamGuard!)

Key Search Terms

When you need to find more resources, use these search terms:

Scikit-Learn: "scikit-learn tutorial python beginners"
Preprocessing: "data preprocessing machine learning python"
Evaluation: "precision recall F1 score explained simply"
Imbalanced data: "handling imbalanced data machine learning"
Train-test: "train test split machine learning overfitting"
General ML: "machine learning for beginners python practical"

After This Phase

Once you're comfortable with:

- Training and evaluating models with Scikit-Learn
- Preprocessing data correctly
- Handling imbalanced data
- Understanding precision, recall, and F1

You're ready for Phase 4: Computer Vision where you'll start working with actual images and video using OpenCV and YOLO. That's where ExamGuard really starts to come alive.